Assignment 5 — One-Dimensional Arrays

Part 1: Multiple Choice (8 Questions · 8 points)

1. For the declaration int scores[8];, which indices are valid?

- ☐ A) 1 through 8
- ☐ B) 0 through 8
- ☐ **C) 0 through 7**
- ☐ D) 0 through 9

2. An int array starts at base address 1000 and each int takes 4 bytes. At what address is arr[5]?

- ☐ A) 1005
- ☐ B) 1016
- ☐ **C) 1020**
- ☐ D) 1024

3. Given: int a[5] = {2, 4}; What are the values of a[1] and a[3]?

- ☐ **A) 4 and 0**
- ☐ B) 4 and garbage (unknown)
- ☐ C) 2 and 4
- ☐ D) It does not compile

4. Which declaration is valid standard C++ (not a non-standard variable-length array)?

- ☐ A) int n; cin >> n; int arr[n];
- ☐ B) int size = 10; int arr[size];
- ☐ **C) const int N = 10; int arr[N];**
- ☐ D) int arr[];

5. What happens if your code writes to arr[10] when arr was declared as int arr[10];?

- ☐ A) A guaranteed compiler error
- ☐ B) The array grows by one element
- ☐ C) A guaranteed clear crash
- ☐ **D) Compiles, but undefined behavior**

6. Given: int n[5] = {4, 9, 15, 22, 7}; int c = 0; for (int i = 0; i < 5; i++) { if (n[i] > 8) c++; } What is c?

- ☐ A) 2
- ☐ **B) 3**
- ☐ C) 4
- ☐ D) 5

7. Which for loop header correctly visits an array of size SIZE from the last element to the first?

- ☐ A) for (int i = SIZE; i > 0; i--)
- ☐ **B) for (int i = SIZE - 1; i >= 0; i--)**
- ☐ C) for (int i = SIZE - 1; i > 0; i--)
- ☐ D) for (int i = 0; i < SIZE; i--)

8. Which statement about passing an array to a function is TRUE?

- ☐ A) A copy is made; original is safe
- ☐ **B) Can change original; const blocks it**
- ☐ C) The size is passed automatically
- ☐ D) Call it as printArray(scores[], 5)

Part 2: Coding Problems (22 points)

Problem 1: Test Score Statistics (7 pts)

Write a C++ program called score_stats.cpp that reads 5 test scores into an array and reports some statistics about them.

- ☐ Declare const int SIZE = 5; and an int array of that size.
- ☐ Use a for loop to read the 5 scores from the user with cin.
- ☐ Use loops to compute the sum, the highest score, and the lowest score (start your max and min from the first element).
- ☐ Compute the average as a double — use static_cast<double> so the decimal part isn't lost.
- ☐ Print all the scores on one line, then the sum, average, highest, and lowest, each clearly labeled.

```
#include <iostream>
using namespace std;

int main() {
   const int SIZE = 5;
   int scores[SIZE];

   for (int i = 0; i < SIZE; i++) {
      cout << "Enter score " << i + 1 << ": ";
      cin >> scores[i];
```

```
    }

    int sum = 0;
    int highest = scores[0];
    int lowest = scores[0];

    for (int i = 0; i < SIZE; i++) {
        sum = sum + scores[i];

        if (scores[i] > highest) {
            highest = scores[i];
        }

        if (scores[i] < lowest) {
            lowest = scores[i];
        }
    }

    double average = static_cast<double>(sum) / SIZE;

    cout << "Scores: ";
    for (int i = 0; i < SIZE; i++) {
        cout << scores[i] << " ";
    }

    cout << endl;
    cout << "Sum: " << sum << endl;
    cout << "Average: " << average << endl;
    cout << "Highest: " << highest << endl;
    cout << "Lowest: " << lowest << endl;

    return 0;
}
```

Problem 2: Search & Count (7 pts)

Write a C++ program called search_count.cpp that searches a fixed array for a number the user chooses.

- ☐ Start with exactly this array: int data[10] = {7, 12, 5, 12, 40, 3, 12, 8, 21, 5}; (use const int SIZE = 10;).
- ☐ Ask the user for a target number using cin.
- ☐ Use a sentinel (foundIndex = -1) to record the index of the FIRST match, and a counter to track how many times the target appears.
- ☐ If found, print the first index and the number of occurrences. If not, print that it was not found.

```
#include <iostream>
using namespace std;

int main() {
    const int SIZE = 10;
    int data[10] = {7, 12, 5, 12, 40, 3, 12, 8, 21, 5};
```

```
    int target;
    cout << "Enter a number to search for: ";
    cin >> target;

    int foundIndex = -1;
    int count = 0;

    for (int i = 0; i < SIZE; i++) {
        if (data[i] == target) {
            count = count + 1;

            if (foundIndex == -1) {
                foundIndex = i;
            }
        }
    }

    if (foundIndex != -1) {
        cout << target << " first found at index "
             << foundIndex << endl;
        cout << target << " appears " << count
             << " time(s)" << endl;
    } else {
        cout << target << " was not found." << endl;
    }

    return 0;
}
```

Problem 3: Array Utilities with Functions (8pts)

Write a C++ program called array_utils.cpp that uses three functions to work with an array passed in from main.

- ☐ In main, create: const int SIZE = 6; int values[SIZE] = {14, 3, 27, 9, 50, 21};
- ☐ Write void printForward(const int arr[], int size) — prints the elements in order.
- ☐ Write void printReverse(const int arr[], int size) — prints the elements from last to first.
- ☐ Write int countAbove(const int arr[], int size, int limit) — returns how many elements are greater than limit. main asks the user for the limit and prints the result.
- ☐ Every function must use its size parameter for its loop bound — no hardcoded 6 inside the functions.

#include <iostream>

```
using namespace std;

void printForward(const int arr[], int size) {
   for (int i = 0; i < size; i++) {
      cout << arr[i] << " ";
   }
   cout << endl;
}

void printReverse(const int arr[], int size) {
   for (int i = size - 1; i >= 0; i--) {
      cout << arr[i] << " ";
   }
   cout << endl;
}

int countAbove(const int arr[], int size, int limit) {
   int count = 0;

   for (int i = 0; i < size; i++) {
      if (arr[i] > limit) {
         count = count + 1;
      }
   }

   return count;
}

int main() {
   const int SIZE = 6;
   int values[SIZE] = {14, 3, 27, 9, 50, 21};

   cout << "Forward: ";
   printForward(values, SIZE);

   cout << "Reverse: ";
   printReverse(values, SIZE);

   int limit;
   cout << "Count values above: ";
   cin >> limit;

   int result = countAbove(values, SIZE, limit);
   cout << "Count: " << result << endl;

   return 0;
```

}
Explanation for codes:

Problem 1: scorestats

This program asks the user to enter 5 test scores and saves them in an array. It uses a for loop to collect the scores and another loop to calculate the sum, highest score, and lowest score. It uses static_cast<double> to calculate the average with decimals. Finally, it prints all the scores and their statistics using cout.

Problem 2: searchcount

This program searches an array of 10 numbers for a number entered by the user. It uses a for loop and an if statement to check each number. The variable foundIndex saves the index where the number first appears, while count tracks how many times it appears. At the end, the program prints the first index and the number of occurrences, or says the number was not found.

Problem 3: arrayutils

This program uses three functions to work with an array of 6 numbers. The printForward function prints the numbers in order, while printReverse prints them backward. The countAbove function counts how many numbers are greater than a limit entered by the user. Finally, main() calls the functions and prints the results.